

Chapter 6 (Limited Direct Execution) — Homework (Measurement)

Zhiyuan Pan

In this homework, you'll measure the costs of a system call and context switch.

Code: [syscall.c](#) and [ctxswitch.c](#). Build with `make`, then run `./syscall`, `./ctxswitch same` and `./ctxswitch diff`.

Test machine: Ubuntu 24.04 VM (VirtualBox 7.2 on an Apple Silicon Mac), Linux 7.0.0 aarch64, 4 vCPUs, gcc 13.3 with `-O2`. Every program was run several times; the numbers below are typical and varied by less than 20% between runs.

1. Timer precision

Measure back-to-back calls to `gettimeofday()` to learn something about how precise the timer really is; this will tell you how many iterations of your null system-call test you'll have to run in order to get a good measurement result.

`syscall.c` calls each timer twice in a row, one million times, and records the smallest non-zero difference.

```
gettimeofday: 978075 of 1000000 back-to-back pairs differ by 0 us;
smallest non-zero step = 1 us
clock_gettime(CLOCK_MONOTONIC): resolution 1 ns; smallest non-zero step
= 41 ns
```

- `gettimeofday()` has a precision of 1 μ s. About 98% of back-to-back calls return exactly the same value, so one call takes well under 1 μ s.

Anything shorter than about 1 μ s cannot be timed with a single measurement.

- **clock_gettime(CLOCK_MONOTONIC) is much finer.** It reports nanoseconds, but the smallest real step is 41 ns. That is one tick of the ARM generic timer, which runs at 24 MHz ($1 / 24 \text{ MHz} \approx 41.7 \text{ ns}$). On ARM this counter plays the role of `rdtsc` on x86, and `clock_gettime` already reads it. I used `clock_gettime` for all the measurements below; `CLOCK_MONOTONIC` also never jumps if the wall-clock time is adjusted.
- **How many iterations to run.** A null system call turns out to take about 150 ns, far below the 1 μ s precision of `gettimeofday`. To keep the timer error under 0.1%, the whole loop has to run for at least about 1 ms ($1 \mu\text{s} / 0.1\%$). I used **1,000,000 iterations** (about 160 ms in total), so the timer error is negligible and both timers would give the same answer.

Side note: on Linux, `gettimeofday()` and `clock_gettime()` do **not** enter the kernel. They run in user space through the vDSO, reading a shared page and the hardware counter. That is why they are so fast, and why they are not suitable as the "null system call" to measure.

2. Cost of a system call

You could repeatedly call a simple system call (e.g., performing a 0-byte read), and time how long it takes; dividing the time by the number of iterations gives you an estimate of the cost of a system call.

`syscall.c` opens `/dev/null` and calls `read(fd, buf, 0)` one million times. A 0-byte read does almost no work, but it is a real system call: it traps into the kernel, looks up the file descriptor, and returns. The cost of an empty loop of the same length is measured separately and subtracted.

```
1000000 x read(fd, buf, 0): 166245 us total, loop overhead 446 us
cost of one system call: 0.166 us (166 ns)
```

run	cost per system call
1	139 ns
2	161 ns

3	162 ns
4	166 ns

A system call costs about 0.14–0.17 μ s ($\approx$ 150 ns). That is roughly 70–80 times more than a plain function call (a few ns), because each call has to trap into the kernel (switching to kernel mode and saving user registers), run the handler, and return from the trap. Running inside a VM adds a bit more on top of that.

3. Cost of a context switch

The lmbench benchmark does so by running two processes on a single CPU, and setting up two UNIX pipes between them ... One difficulty in measuring context-switch cost arises in systems with more than one CPU; what you need to do on such a system is ensure that your context-switching processes are located on the same processor ... on Linux, for example, the `sched_setaffinity()` call is what you're looking for.

`ctxswitch.c` copies the lmbench method:

1. Create two pipes: `p1` (parent $\rightarrow$ child) and `p2` (child $\rightarrow$ parent).
2. `fork()`. Both processes call `sched_setaffinity()` to pin themselves to **CPU 0**.
3. The parent writes one byte into `p1`, then blocks reading `p2`. The OS has to switch to the child, which reads from `p1`, writes into `p2`, and blocks reading `p1` again. The OS switches back to the parent, and the cycle repeats.
4. Each round trip therefore contains **2 context switches** (parent $\rightarrow$ child $\rightarrow$ parent). The program times 200,000 round trips.

```
same CPU: 200000 round trips in 425541 us
  per round trip: 2.128 us
  per context switch (incl. one pipe write + read): 1.064 us
different CPU: 200000 round trips in 10285447 us
  per round trip: 51.427 us
  per context switch (incl. one pipe write + read): 25.714 us
```

run	round trip (same CPU)	per switch (same CPU)	per switch (different CPUs)
1	2.071 µs	1.036 µs	26.2 µs
2	2.027 µs	1.013 µs	25.9 µs
3	2.004 µs	1.002 µs	25.8 µs
4	2.128 µs	1.064 µs	25.7 µs

With both processes on the same CPU, one context switch costs about 1.0 µs. That number still includes one pipe `write` and one pipe `read` per switch. Each round trip makes 4 system calls ($\approx 4 \times 0.15 \mu s = 0.6 \mu s$ from section 2, and pipe calls cost somewhat more than a 0-byte read). Subtracting them leaves roughly **0.7 µs for the switch itself**: saving one process's registers, running the scheduler, switching address spaces, and restoring the other process. That is about 5 times the cost of a system call.

Why pin both processes to one CPU? The `diff` run puts the child on CPU 1 and is about **25 times slower** (26 µs per "switch"). With two CPUs, no context switch happens on either CPU. Instead, each side goes idle while it waits, and every byte has to *wake up* the other CPU with an inter-processor interrupt. In a VM this is especially expensive, because an idle vCPU is a sleeping host thread that the hypervisor must wake up. That measures CPU wake-up latency, not context-switch cost, which is exactly why the book says to use `sched_setaffinity()`.

Summary

operation	measured cost
<code>gettimeofday()</code> precision	1 µs

<code>clock_gettime()</code> precision	41 ns (24 MHz counter)
null system call (<code>read</code> of 0 bytes)	$\approx 0.15\ \mu\text{s}$
context switch, same CPU (incl. pipe I/O)	$\approx 1.0\ \mu\text{s}$
context switch, same CPU (pipe I/O subtracted)	$\approx 0.7\ \mu\text{s}$
ping-pong across two CPUs	$\approx 26\ \mu\text{s}$ per direction (wake-up latency)

Limitations: the measurements were taken inside a virtual machine, so the hypervisor adds some overhead, especially for anything involving interrupts or idle CPUs. The context-switch number also does not include the *indirect* cost of a switch: after switching, the caches and TLB are "cold" for the new process. Because these two processes touch almost no memory, that cost barely shows up here.